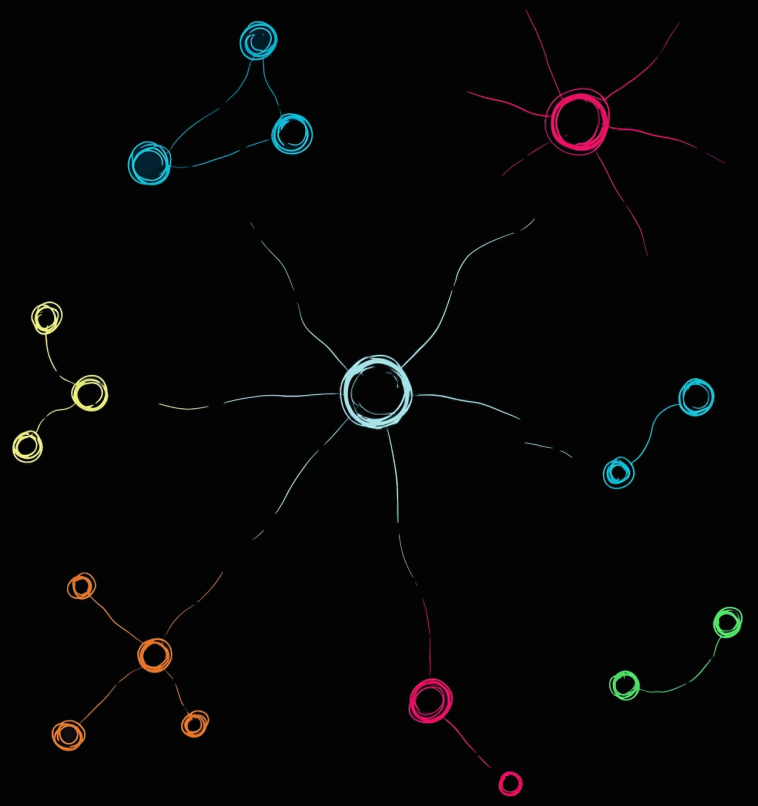


Neyron Şəbəkələrinin Əsasları

Perseptrondan optimizer-ə qədər — süni intellektin ürəyinə səyahət

Kurs Planı

Bu təqdimatda neyron şəbəkələrinin bütün əsas bloklarını addım-addım öyrənəcəyik.

01

Süni neyron və activation functions

Beyindən riyaziyyata: w, b, f(z)

02

Birqatlı perseptron

Rosenblatt, 1958 — ilk öyrənən maşın

03

Çoxqatlı perseptron (MLP)

XOR problemi, hidden layers, representation

04

Forward propagation və Loss

Proqnoz vermə, MSE, Cross-Entropy

05

Backpropagation

Chain rule, gradient-lər, vanishing gradient

06

Optimization və optimizer-lər

SGD, Momentum, RMSprop, Adam



Süni Neyron

İnsan beynində təxminən **86 milyard neyron** var. Süni neyron bunun çox sadələşdirilmiş, lakin güclü riyazi versiyasıdır.

Düstur

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

$$\hat{y} = f(z)$$

- **x** — giriş siqnalı
- **w** — weight (girişin vacibliyi)
- **b** — bias (ümumi "əhval-ruhiyyə")
- **f** — activation function

Misal: "Pizza yeməyə gedim?"

$x_1 \cdot w_1$

Hava yaxşıdır — orta çəki

$x_2 \cdot w_2$

Dostlar gəlir — **ən böyük çəki!**

$x_3 \cdot w_3$

Pul var — orta çəki

Niyə Activation Function Lazımdır?

Çox tez-tez verilən sual: "Niyə sadəcə xətti hesablama etmirik?" Cavab çox vacibdir.

Aktivasiya olmasa...

Neçə qat qoysaq da, şəbəkə yenə xətti funksiya olaraq qalır:

$$W_2(W_1 x) = (W_2 W_1)x = W'x$$

Yüz qat = bir qat! Heç bir fərq yoxdur.

Aktivasiya ilə...

Qeyri-xəttilik (non-linearity) məhz *f* sayəsində gəlir. Şəbəkə artıq əyri, mürəkkəb sərhədlər öyrənə bilir.

- Spiralları ayıra bilir
- Üzləri tanıya bilir
- Dili anlaya bilir

Bütün dərin öyrənmənin sirri burada gizlənir.

Əsas Activation Functions

Funksiya	Düstur	Aralıq	Nə vaxt?
Step	1 əgər z≥0, əks halda 0	{0,1}	Klassik perseptron
Sigmoid	$1 / (1 + e^{-z})$	(0, 1)	Binary classification çıxışı
Tanh	$(e^{z} - e^{-z}) / (e^{z} + e^{-z})$	(−1, 1)	Hidden layer (köhnə üslub)
ReLU	max(0, z)	[0, ∞)	**Default hidden layer seçimi**
Leaky ReLU	max(0.01z, z)	(−∞, ∞)	"Ölü neyron" problemi olanda
Softmax	$e^{zi} / \Sigma e^{zj}$	(0, 1), cəm=1	Multi-class çıxışı

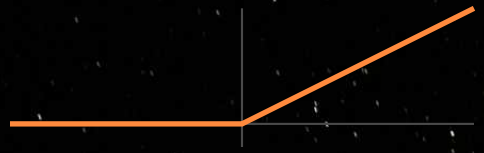
Sigmoid-in törəməsi heç vaxt 0.25-dən böyük olmur — bu vanishing gradient probleminin əsas səbəbidir.

Activation Functions — Sadə Misallarla

Neyron girişləri çəkilərlə toplayıb bir ədəd alır, məsələn **z = 5**. Activation function bir suala cavab verir: **neyron bu ədədlə nə etsin** və növbəti qata hansı siqnalı ötürsün?

ReLU

«Açar» (switch)



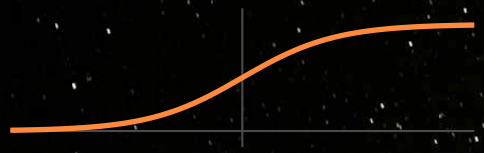
Mənfi ədəd → **0**, müsbət ədəd → olduğu kimi qalır.

−7 → 0
−1 → 0
3 → 3
10 → 10

Yalnız «hə»-yə reaksiya verən sensor kimi. Ən populyar seçim: sadədir və yaxşı işləyir.

Sigmoid

«0-dan 1-ə inam»



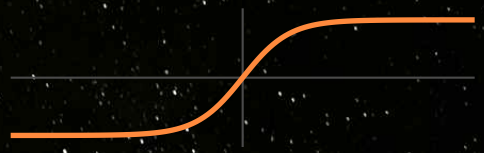
İstənilən ədədi ehtimala bənzər qiymətə çevirir.

−10 → ≈0 «qətiyyən yox»
0 → 0.5 «bilmirəm»
2 → 0.88 «yəqin ki, hə»
10 → ≈1 «mütləq hə»

Çıxışda əlverişlidir: «bu məktub 88% ehtimalla spamdır».

Tanh

«−1-dən 1-ə şkala»



Sigmoid-ə oxşayır, amma nəticə −1 ilə 1 arasındadır.

−10 → −1
0 → 0
10 → 1

«Əleyhinə — neytral — lehinə» şkalası kimi düşünün.

Softmax

«100%-i bölüşdür»



Şəklin balları: pişik 4, it 2, quş 1.

pişik → ≈84%
it → ≈11%
quş → ≈4%

Cəm həmişə 100%-dir; ən böyük bal əsas payı alır.

Niyə ReLU Sigmoid-i əvəz Etdi?

Sigmoid-in problemi

Böyük ədədlərdə sigmoid 0-a və ya 1-ə «dirənir» və demək olar ki, dəyişmir. Şəbəkə isə kiçik dəyişikliklərin nəticəyə necə təsir etdiyini görərək öyrənir. Dəyişiklik yoxdursa, öyrənmə dayanır.

Sigmoid: 10 → ≈1, 20 → ≈1 — fərq yoxdur

ReLU: 10 → 10, 20 → 20 — **fərq həmişə görünür**

Analogiya

Səs düyməsi artıq sona qədər burulmuş kalonka kimi: düyməni bursan, amma səs dəyişmir.

Yekun: hansını harada?

Harada?	Hansı funksiya?
Gizli qatlar (hidden layers)	**ReLU** və ya GELU kimi «hamar qohumları»
Çıxış: hə / yox	Sigmoid
Çıxış: bir neçə sinifdən biri	Softmax
Çıxış: ədəd (regression)	Activation yoxdur (linear)

Birqatlı Perseptron

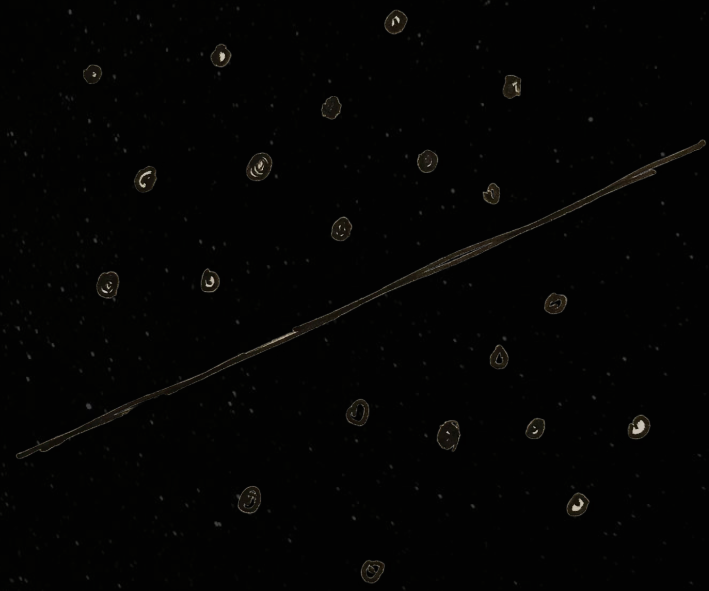
FRANK ROSENBLATT, 1958

Tarixdə ilk öyrənən maşın: bir neyron + step funksiyası. Quruluşu sadədir, amma arxasında böyük fikir var.

$$\hat{y} = \begin{cases} 1 & \text{əgər } \mathbf{w} \cdot \mathbf{x} + b \geq 0 \\ 0 & \text{əks halda} \end{cases}$$

Həndəsi məna

w·x + b = 0 tənliyi 2D-də düz xətt, 3D-də müstəvi verir. Perseptron məlumatı bir düz xətlə iki hissəyə bölür — bu onun həm gücü, həm də məhdudiyyətidir.



Perseptron Necə Öyrənir?

Öyrənmə qaydası sadədir: **düzgün edirsən — heç nə etmə; yanlış edirsən — düzəlt.**

1

Proqnoz ver

Forward pass: $\hat{y} = \text{step}(w \cdot x + b)$

2

Səhvi yoxla

Əgər $\hat{y} = y \rightarrow$ heç nə etmə

3

Çəkiləri yenilə

$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$

$b \leftarrow b + \eta(y - \hat{y})$

4

Nəticə

AND/OR-u 2–3 epoch-da öyrənir, accuracy 100%

Burada **η (eta)** — learning rate, hər addımın böyüklüyüdür. Çox kiçik olsa yavaş öyrənir, çox böyük olsa "sıçrayır".

XOR Problemi — Perseptronun Divarı

XOR cədvəli

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

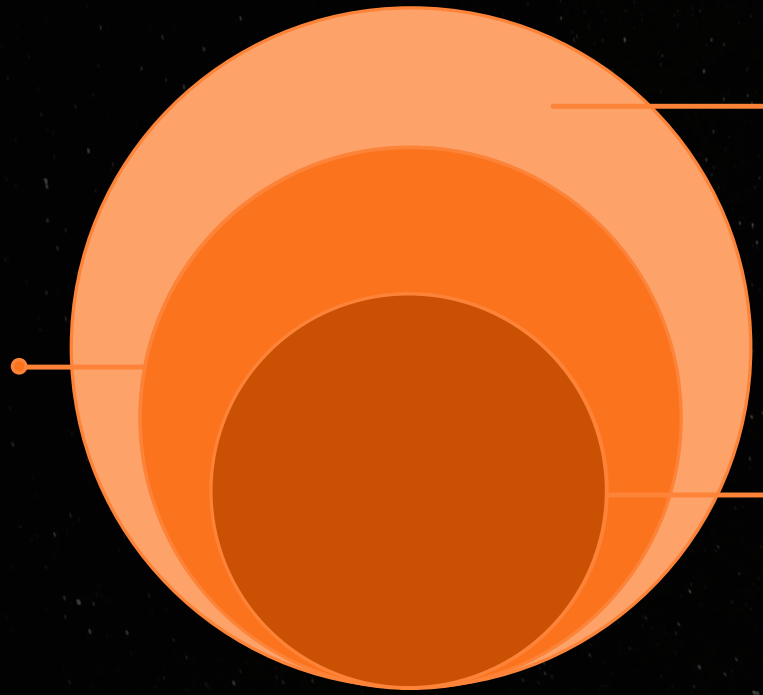
Niyə perseptron bacarmır?

Heç bir **düz xətt** bu dörd nöqtəni düzgün ayıra bilmir. Perseptron yalnız *linearly separable* məsələləri həll edə bilir.

1969: Minsky və Papert bunu rəsmi isbat etdi → "AI Winter" başladı. Çıxış yolu: bir neçə qat və bir neçə neyron!

Çoxqatlı Perseptron (MLP)

Cavab sadədir: qatları artır! Hər qat məlumatı yeni bir "baxış bucağından" görür.



Çıxış Qatı
2 neyron: nəticə y1 y2

Gizli Qat
4 neyron: nümunələri öyrənir

Giriş Qatı
3 neyron: x1 x2 x3 xam məlumat

Input Layer
Xam məlumatı qəbul edir. Heç bir hesablama yoxdur.

Hidden Layer(s)
Əsas "düşünmə" burada gedir. Representation öyrənilir.

Output Layer
Son cavab buradan çıxır. Activation məsələyə görə seçilir.

Hər neyron əvvəlki qatın **bütün** neyronları ilə bağlıdır → *fully connected* və ya *dense* qat adlanır.

MLP XOR-u Necə Həll Edir?

Gizli qatda 2 neyron

$h_1 = OR(x_1, x_2)$

$0 \to 0, 0 \to 1, 1 \to 1, 1 \to 1$

$h_2 = NAND(x_1, x_2)$

$1 \to 1, 1 \to 1, 1 \to 1, 0 \to 0$

$y = AND(h_1, h_2)$

$\to 0, 1, 1, 0$ ✓

Əsas fikir

Gizli qat məlumatı **yeni fəzaya köçürür**. Orijinal fəzada ayrılmaz olan məlumat, (h_1, h_2) fəzasında bir düz xətlə ayrıla bilir.

Düz xətt: $\mathbf{h_1 + h_2 = 1.5}$ — kifayətdir!

Dərin öyrənmənin bütün gücü bu fikirdədir: hidden layer-lər məlumatın *representation*-ını dəyişir.

Şəbəkənin Ölçüsü və Universal Approximation

Parametr sayını hesabla

Bir dense qatda: **n_in × n_out** çəki + **n_out** bias.

Qat	Çəki	Bias
2 → 4	8	4
4 → 4	16	4
4 → 1	4	1
Cəmi	**28**	**9 = 37**

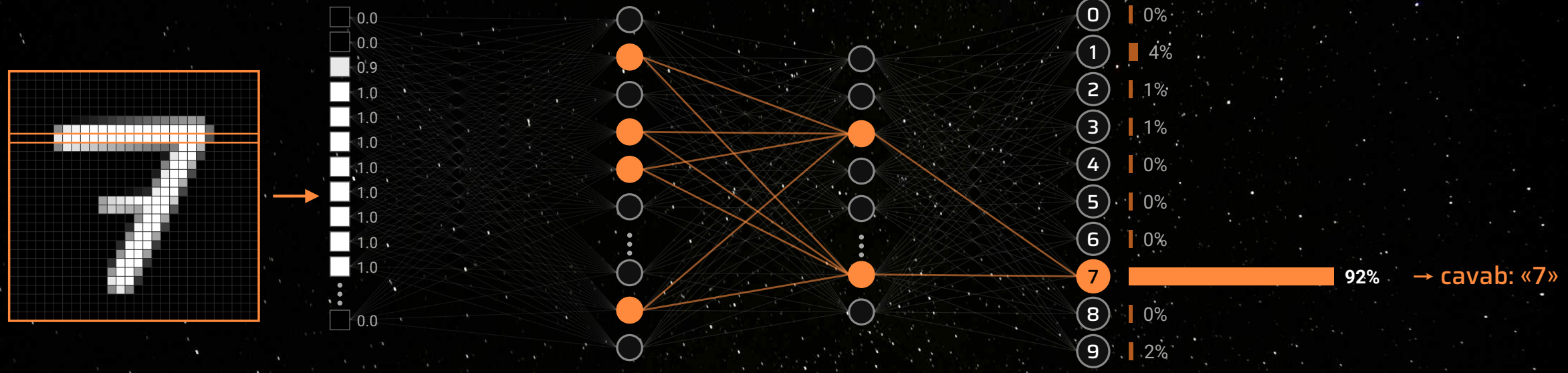
Universal Approximation Teoremi

Bir gizli qatı olan və **kifayət qədər neyronu** olan MLP, istənilən kəsilməz funksiyanı istənilən dəqiqliklə yaxınlaşdıra bilər.

Amma bu teoremin mənası odur ki, *belə şəbəkə mövcuddur* — doğru çəkiləri tapmaq ayrıca, çox daha çətin məsələdir!

MLP Rəqəmi Necə Tanıyır?

Klassik nümunə — **MNIST**: əllə yazılmış rəqəmlər. Şəbəkə şəkli ədədlərə çevirir və hansı rəqəm olduğunu təxmin edir.



Giriş şəkli
28 × 28 = 784 piksel

784 ədəd
0 = qara, 1 = ağ

Gizli qat 1
128 neyron · ReLU

Gizli qat 2
64 neyron · ReLU

Çıxış qatı
10 neyron · Softmax

1 Piksellər → ədədlər

Hər piksel 0 ilə 1 arasında ədəddir. Şəkil 784 ədədlik vektora çevrilir.

2 Gizli qat 1

Neyronlar sadə izləri tutur: kənarlar, kiçik xətt parçaları.

3 Gizli qat 2

İzləri birləşdirir: «yuxarıda üfüqi xətt + aşağı maili xətt».

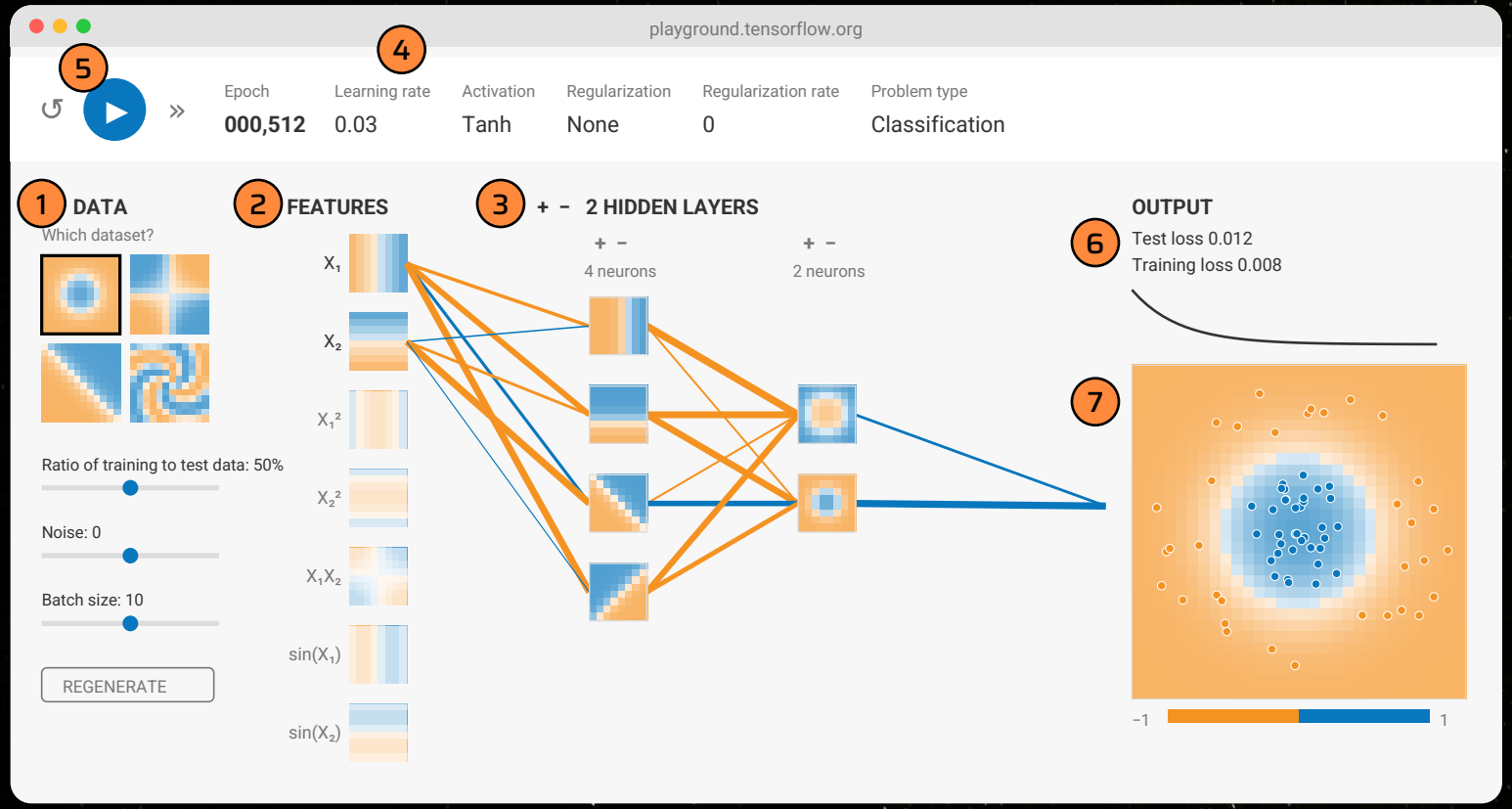
4 Softmax çıxışı

10 neyron = 10 rəqəm. Ən böyük ehtimal qalib gəlir: «7» — 92%.

Parametr sayı: (784·128 + 128) + (128·64 + 64) + (64·10 + 10) = **109 386**. Belə sadə MLP MNIST-də təxminən 97–98% dəqiqlik verir. Qatların «nə gördüyü» sadələşdirilmiş təsəvvürdür.

TensorFlow Playground: İnterfeys

Sayt: **playground.tensorflow.org** — quraşdırma tələb etmir, brauzerdə işləyir. Ekran 4 hissədən ibarətdir: data, giriş, şəbəkə və nəticə.



1 **DATA**
Dataset seçimi (Circle, XOR, Gaussian, Spiral), train/test nisbəti, noise, batch size.

2 **FEATURES**
Şəbəkənin girişləri. Kliklə yandırıb-söndürülür; adətən yalnız X_1 və X_2.

3 **HIDDEN LAYERS**
+ / − ilə qat və neyron sayı. Hər kvadrat bir neyronun «gördüyüdür».

4 **Yuxarı panel**
Learning rate, Activation, Regularization, Problem type.

5 ↺ ▶ »
Sıfırla, öyrənməni başlat/dayandır, bir addım irəli. Epoch sayğacı yanındadır.

6 **Loss**
Test loss və Training loss + loss-un qrafiki.

7 **Qərar sərhədi**
Fon rəngi — şəbəkənin proqnozu, nöqtələr — data.

Playground: İlk Təcrübə Addım-addım

Nümunə: **Circle** datasetini 1 gizli qatla həll edək. Hər təcrübə eyni 6 addımdan ibarətdir.

1 **Dataset seç**
DATA bölməsində **Circle**-a klikləyin. Noise = 0, ratio = 50% saxlayın.

2 **Girişləri seç**
FEATURES-də yalnız X_1 və X_2 aktiv qalsın, qalanlarını söndürün.

3 **Şəbəkəni qur**
+ ilə **1 gizli qat** əlavə edin, neyron sayını **4**-ə çatdırın.

4 **Parametrləri təyin et**
Learning rate = **0.03**, Activation = **Tanh**, Regularization = None.

5 **↺, sonra ▶**
Əvvəl sıfırlayın, sonra başladın. ~300 epoch sonra eyni düymə ilə dayandırın.

6 **Nəticəni oxu**
Test loss < 0.05 olmalıdır. Fon mavi dairəni əhatə edirsə — şəbəkə öyrəndi.

Rəngləri necə oxumalı?

Mavi — müsbət dəyər (+1 sinfi)

Narıncı — mənfi dəyər (−1 sinfi)

Qalın xətt — çəkinin (weight) böyüklüyü

Solğun fon — şəbəkə əmin deyil (≈ 0)

Test loss vs Training loss

Training loss — öyrəndiyi data üzərində səhv. **Test loss** — heç görmədiyi data üzərində səhv. Training aşağı, test yuxarıdırsa — **overfitting**.

İpucu: neyron kvadratının üzərinə siçanı gətirin — onun «gördüyü» sağda böyük göstəriləcək. Xəttin üzərinə gətirsəniz, çəkinin dəqiq qiymətini görəcəksiniz.

TensorFlow Playground

Nəzəriyyəni brauzerdə canlı sınayın: **playground.tensorflow.org**

1. **XOR müqayisəsi**

   Hidden layer olmadan və olduqda accuracy-ni müqayisə et

2. **Activation dəyişdir**

   Linear, ReLU, Tanh — hər birinin qərar sərhədini gör

3. **Spiral dataset**

   2–3 qat, 6–8 neyron qur — mürəkkəb həndəsəni öyrən

4. **Learning rate = 3**

   Loss-un "partlamasını" canlı izlə — divergensiya nədir gör

5. **Overfitting**

   Noise artır, böyük şəbəkə qur — train/test fərqini müşahidə et

Forward Propagation

Məlumat şəbəkədən **soldan sağa** keçib proqnoza çevrilir. Hər qat eyni iki addımı icra edir:

$$z^{[l]} = W^{[l]} \cdot a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = f\left(z^{[l]}\right)$$

Qeyd

- **$a^{(0)} = x$** — ilk giriş
- **$a^{(L)} = \hat{y}$** — son çıxış
- f — həmin qatın activation function-ı

W matrisinin ölçüsü

Əvvəlki qatda **n**, bu qatda **m** neyron varsa:

$$W \in \mathbb{R}^{m \times n}$$

Ölçülərə diqqət etmək çoxlu xətanın qarşısını alır!

Forward Propagation: Ədədi Nümunə

ŞƏBƏKƏ: 2 → 2 → 1, HƏR YERDƏ SIGMOID

Verilənlər

x = (1.0, 0.5)

$\mathbf{W^1}$ = [[0.2, 0.4], [−0.5, 0.1]], $\mathbf{b^1}$ = (0.1, −0.2)

$\mathbf{W^2}$ = [0.7, −0.3], $\mathbf{b^2}$ = 0.05

Hesablama

1-ci qat

$z_1 = 0.2 \cdot 1.0 + 0.4 \cdot 0.5 + 0.1 = \mathbf{0.5}$

$z_2 = -0.5 \cdot 1.0 + 0.1 \cdot 0.5 - 0.2 = \mathbf{-0.65}$

$a^1 \approx \text{sigmoid}(0.5, -0.65) \approx (0.622, 0.343)$

2-ci qat

$z_2 = 0.7 \cdot 0.622 + (-0.3) \cdot 0.343 + 0.05 \approx \mathbf{0.383}$

$\hat{y} = \mathbf{sigmoid(0.383)} \approx \mathbf{0.595}$

Loss Function

Loss şəbəkənin nə qədər səhv etdiyini göstərən tək rəqəmdir. Öyrənmənin hədəfi bu rəqəmi minimuma endirməkdir.

Regression üçün — MSE

$$L = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})^2$$

Binary Classification üçün — BCE

$$L = -\frac{1}{m}\sum\left[y\log\hat{y} + (1-y)\log(1-\hat{y})\right]$$

Niyə BCE güclüdür?

−log(ŷ) əyrisi düşünün: şəbəkə **əmin şəkildə yanlış** cavab verdikdə (məsələn, ŷ=0.01, y=1), loss çox böyük olur — güclü "cəza".

Bu şəbəkəni tez düzəlməyə məcbur edir.

Multi-class məsələlər üçün Categorical Cross-Entropy + Softmax istifadə edilir.

Backpropagation: Əsas İdeya

Sual: hər çəkini nə qədər dəyişsək, loss azalar? Yəni **hər çəki üçün gradient lazımdır: dL/dw**.

Chain Rule

$$\frac{dL}{dw} = \frac{dL}{da} \cdot \frac{da}{dz} \cdot \frac{dz}{dw}$$

Hər şəbəkə qatı üçün bu zəncir geriyə doğru tətbiq edilir.

Analogiya

Layihə uğursuz oldu. "Kim nə qədər günahkardır?" sualını **son mərhələdən başlayıb geriyə doğru** araşdırırıq.

Son qat ən birbaşa günahkardır. İlk qatlara çatana qədər "məsuliyyət" paylaşdırılır.



Backpropagation: Düsturlar

Δ⁽ˡ⁾ = DL/DZ⁽ˡ⁾ — QATIN "XƏTA SIQNALI"

Son qat

$$\delta^{[L]} = \frac{dL}{da^{[L]}} \odot f'\left(z^{[L]}\right)$$

Geriyə doğru (hər qat)

$$\delta^{[l]} = \left(W^{[l+1]T}\delta^{[l+1]}\right) \odot f'\left(z^{[l]}\right)$$

Gradient-lər

$$\frac{dL}{dW^{[l]}} = \delta^{[l]} \cdot a^{[l-1]T}, \quad \frac{dL}{db^{[l]}} = \delta^{[l]}$$

Tam Alqoritm

01

Forward pass

Bütün z⁽ˡ⁾ və a⁽ˡ⁾ saxla

02

Loss hesabla

L(ŷ, y) dəyərini tap

03

Backward pass

δ-ları L-dən x-ə qədər say

04

Update et

W ← W − η · dL/dW

Backpropagation: Yoxlama və Problem

Gradient Check

$$\frac{dL}{dw} \approx \frac{L(w+\varepsilon) - L(w-\varepsilon)}{2\varepsilon}$$

Əl ilə yazdığımız backprop ilə fərq ≈ **1e-11** çıxdı — düzgündür!

Bu üsul debug etmək üçün əla vasitədir.

Vanishing Gradient Problemi

Sigmoid-in törəməsi maksimum **0.25**-dir.

$$0.25^{10} \approx 10^{-6}$$

10 qatdan sonra gradient demək olar ki, yox olur — ilk qatlar öyrənmir!

Həll

- **ReLU** — törəməsi ya 0, ya 1
- **He / Xavier initialization**
- Batch Normalization

MLP-ni Sıfırdan Yazdıq (NumPy)

Hidden qatda **tanh**, çıxışda **sigmoid**, loss: Binary Cross-Entropy. Sigmoid + BCE birlikdə olduqda $\delta^{(L)} = \hat{y} - y$ — çox sadə!

"Two Moons" Dataset Nəticəsi

97.7%

Accuracy

2 qat × 8 neyron ilə

37

Parametr

2→4→4→1 şəbəkəsi

Müqayisə

1 gizli neyron

Demək olar düz xətt — zəif

4 neyron

Yaxşı əyilir, kifayətdir

2 qat × 16

Rahat öyrənir, amma overfitting riski var!

Optimization və Gradient Descent

Optimization — loss L(θ)-nı minimuma endirən parametrləri tapmaqdır.

$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$

Analogiya

Sis içində dağdasan. Yalnız ayağının altındakı yamacı hiss edirsən. Ən sıldırım **aşağı** istiqamətdə addım at — və təkrarla.

Learning Rate seçimi

η = 0.005

Çox kiçik — çox yavaş

η = 0.1

Optimal — hamar eniş

η = 1.05

Çox böyük — divergensiya!



Batch, SGD, Mini-batch

Hər dəfə neçə nümunəyə baxaraq gradient hesablamalıyıq? Bu sualın cavabı praktikada çox vacibdir.

Üsul	Nümunə sayı	Üstünlük	Çatışmazlıq
Batch GD	Bütün dataset	Sabit, hamar yol	Böyük data-da çox yavaş
Stochastic GD	1 nümunə	Çox sürətli update	Əsəbi, titrək yol
Mini-batch GD	**16–256 nümunə**	İkisinin yaxşı tərəfi	Batch size seçimi lazımdır

Bütün training data-nın bir tam keçidinə **epoch** deyilir. Praktikada ən çox mini-batch GD istifadə edilir.

Momentum və RMSprop

Momentum

$$v \leftarrow \beta v + \nabla L$$

$$\theta \leftarrow \theta - \eta v$$

β ≈ 0.9. Top yamacdan aşağı yuvarlanır — hər addımda sürət toplayır. Dar dərələrdə daha sürətli keçir.

RMSprop

$$s \leftarrow \rho s + (1-\rho)(\nabla L)^2$$

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{s+\varepsilon}} \nabla L$$

Hər parametrin **öz fərdi learning rate-i** olur. Böyük gradient-lər yavaşlayır, kiçiklər sürətlənir.

Bu iki ideyanı birləşdirəndə nə alırıq? **Adam optimizer!**

Adam Optimizer

MOMENTUM + RMSPROP — BU GÜN ƏN ÇOX IŞLƏDILƏN DEFAULT

Düsturlar

$$m \leftarrow \beta_1 m + (1 - \beta_1)\nabla L$$

$$v \leftarrow \beta_2 v + (1 - \beta_2)(\nabla L)^2$$

$$\hat{m} = \frac{m}{1 - \beta_1^t}, \quad \hat{v} = \frac{v}{1 - \beta_2^t}$$

$$\theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon}$$

Standart Hiperparametrlər

β_1 = 0.9

Momentum faktoru

β_2 = 0.999

RMSprop faktoru

ε = 1e-8

Sıfıra bölməmək üçün

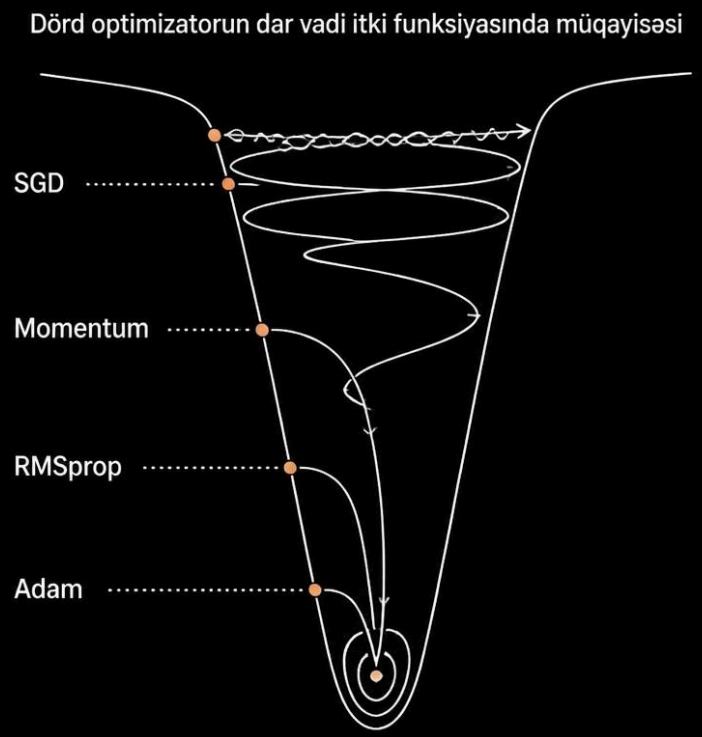
η = 0.001

Default learning rate

m̂ və v̂ — başlanğıcda bias-ı düzəltmək üçün **bias correction**.

Optimizer-lərin Müqayisəsi

Dar dərə funksiyası **f(x,y) = 0.05x² + y²** üzərində dörd optimizer-in yolu:



SGD
vadi divarlarında dalğalanaraq qalıb

Momentum
həddindən artıq ötür amma konvergensiyaya çatır

RMSprop
demək olar ki, düz xətlə minimuma gedir

Adam
sürətlidir amma bir qədər həddindən artıq ötür

Yekun sıralama:
SGD ən yavaşdır,
Momentum və Adam sürətli və hamar,
RMSprop real MLP-də bir qədər səs-küylüdür.

SGD Ən yavaş, dərədə ilişir	**Momentum** Sürətli, bir az ötüb keçir
RMSprop Demək olar düz, real MLP-də sıçrayışlı	**Adam** Sürətli, hamar, **default seçim**

"Ən yaxşı optimizer" yoxdur — hiperparametrlər və məsələ növü çox vacibdir.



Tez-Tez Rast Gəlinən Problemlər

Local Minimum / Saddle Point

Loss düzənliyə çatır, gradient≈0 amma optimal deyil. Momentum və Adam çox vaxt çıxış yolu tapır.

Exploding Gradients

Gradient çox böyüyür, NaN görünür. **Həll:** gradient clipping — ∇L-ni sabit dəyərlə kəs.

Yanlış Learning Rate

Çox kiçik → yavaş. Çox böyük → divergensiya. LR schedule və ya Adam istifadə et.

Overfitting

Train loss aşağı, test loss yuxarı. **Həll:** daha çox data, L2 regularization, Dropout, Early Stopping.

Praktikada: Keras ilə 3 Sətir

Bütün forward propagation, backpropagation və optimizer bu sətirlərin içinə gizlənib. Siz yalnız arxitektura seçirsiniz:

```
from tensorflow import keras

model = keras.Sequential([
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(1,  activation='sigmoid')
])

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=100, batch_size=32)
```

Bu kod "two moons" datasetini ≈98% accuracy ilə həll edir. Əvvəl sıfırdan yazdığımız şəbəkə ilə eyni nəticə!

Xülasə: Böyük Mənzərə



İrəli yayımlanma

Zərər hesabı

Geri yayılma

Optimizator yeniləməsi

Təkrarla

Neyron

$\hat{y} = f(w \cdot x + b)$

Activation

Qeyri-xəttilik gətirir

Perseptron

Bir düz xətt, XOR yoxdur

MLP

Gizli qatlar = yeni fəza

Adam

Momentum + RMSprop = default

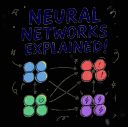
Əlavə Mənbələr

Daha dərindən öyrənmək istəyənlər üçün ən yaxşı mənbələr:



TensorFlow Playground

Brauzerdə canlı eksperiment. Ən yaxşı başlanğıc nöqtəsi. **playground.tensorflow.org**



3Blue1Brown — "But what is a neural network?"

Vizual, intuitiv, YouTube-da pulsuz. Riyaziyyatı animasiya ilə izah edir.



Michael Nielsen — "Neural Networks and Deep Learning"

Pulsuz onlayn kitab. Hər konsept Python kodu ilə izah edilir.



Stanford CS231n Qeydləri

Dərin öyrənmənin ən məşhur açıq kursu. Hər dərsin qeydləri onlayn mövcuddur.